Nimbo Data Ingester - Complete Technical Report

1. Executive Summary

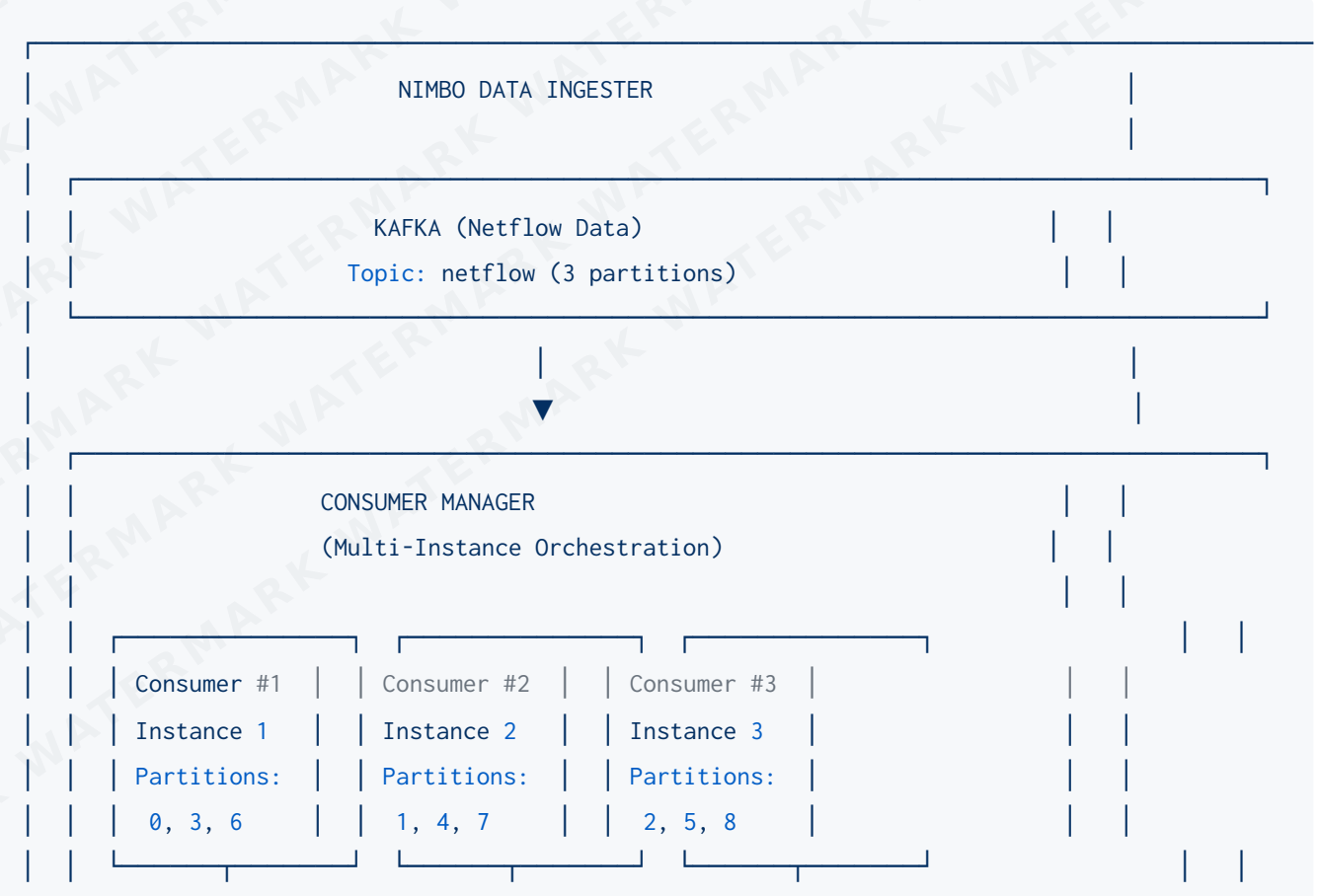
The Nimbo Data Ingester is a production-grade, horizontally scalable data ingestion service that consumes Netflow records from Apache Kafka in Avro format, deserializes them, and persists them as columnar Parquet files to HDFS with zero data loss guarantees. The system implements a multi-consumer architecture with parallel writer threads, providing both horizontal and vertical scalability while maintaining exactly-once processing semantics.

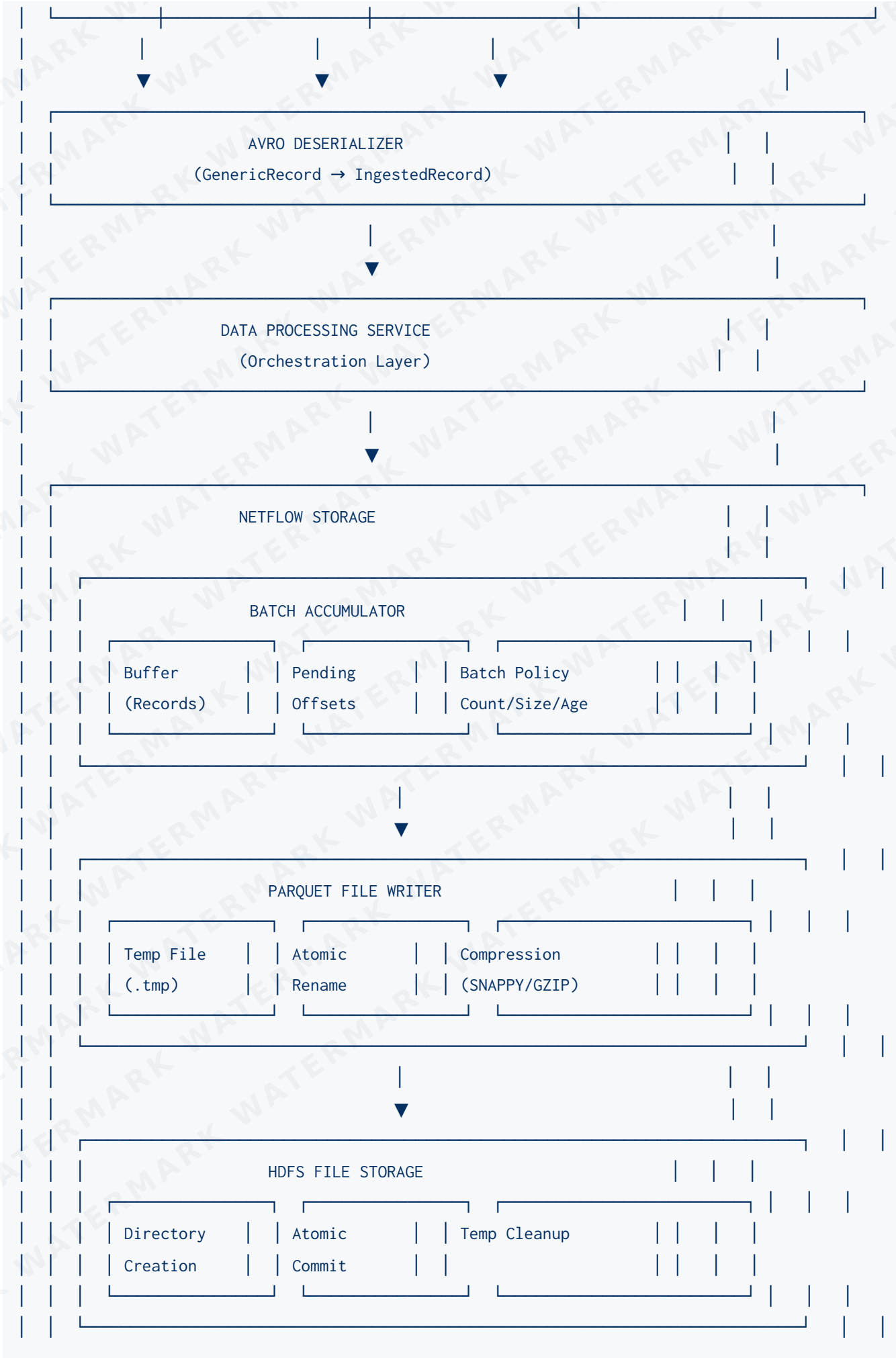
This report documents the complete implementation of the data processing pipeline, including the Kafka consumer, Avro deserialization, batch accumulation, HDFS storage, and Parquet writing components.

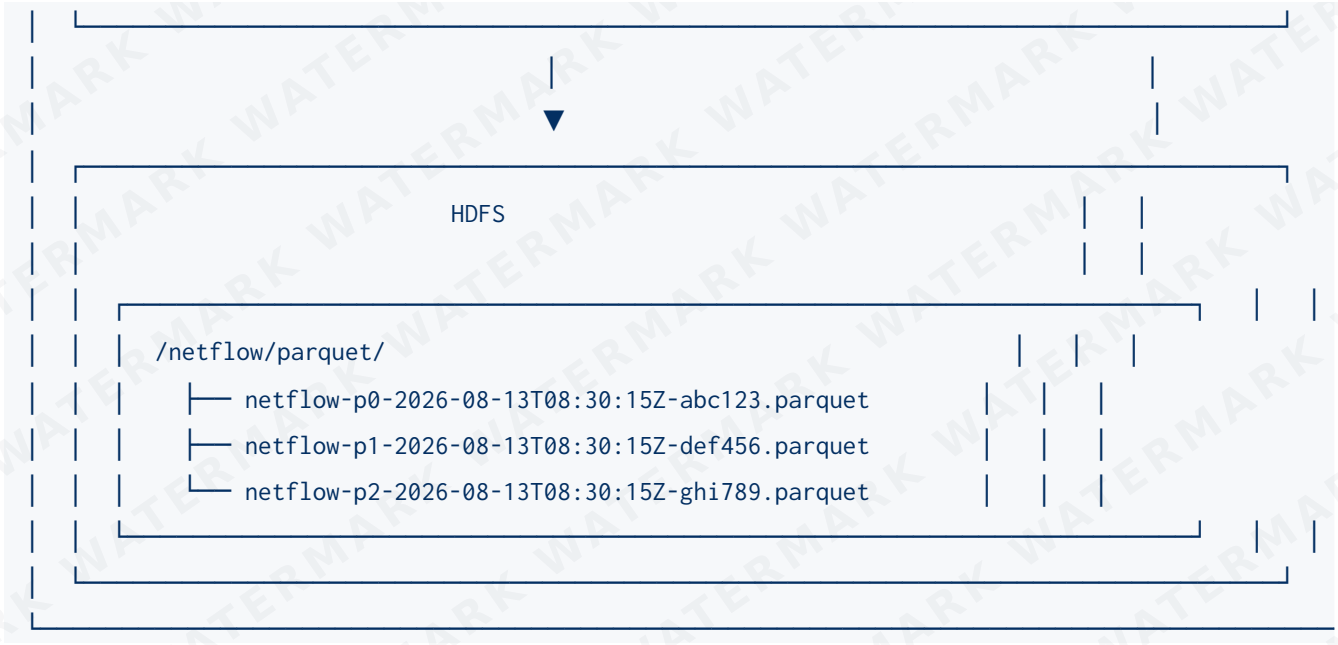
---

2. System Architecture Overview

2.1 High-Level Architecture




AVRO DESERIALIZER
(GenericRecord → IngestedRecord)
DATA PROCESSING SERVICE
(Orchestration Layer)
NETFLOW STORAGE
BATCH ACCUMULATOR
Buffer
(Records)
Pending
Offsets
Batch Policy
Count/Size/Age
PARQUET FILE WRITER
Temp File
(.tmp)
Atomic
Rename
Compression
(SNAPPY/GZIP)
HDFS FILE STORAGE
Directory
Creation
Atomic
Commit
Temp Cleanup



2.2 Key Design Principles

Principle	Implementation
Zero Data Loss	Manual offset commits only after HDFS persistence
Horizontal Scalability	Multiple consumer instances with Kafka consumer groups
Vertical Scalability	Multi-threaded writer pools per consumer
Exactly-Once Semantics	Atomic file operations with temp file + rename
Fault Tolerance	Graceful shutdown with flush on termination
Batch Processing	Accumulate records before writing for efficiency
Configuration Externalization	All configs via YAML + environment variables

3. Module 1: Kafka Consumer (Ingester)

3.1 ConsumerManager - Multi-Instance Orchestration

File: `src/main/java/com/nimbo/ingester/consumer/ConsumerManager.java`

The `ConsumerManager` is responsible for orchestrating multiple consumer instances. It creates and manages a configurable number of consumer threads, each running in its own JVM thread with a unique instance ID.

Key Features:

- Configurable number of consumer instances via `consumer-instances` property
- Thread pool for managing consumer lifecycle
- Graceful shutdown with `@PreDestroy`
- Each consumer receives a unique instance ID for tracking

```
@Component
public class ConsumerManager {
    @PostConstruct
    public void start() {
        int instanceCount = kafkaProperties.getConsumerInstances();
        for (int i = 0; i < instanceCount; i++) {
            NetflowConsumer consumer = new NetflowConsumer(
                kafkaProperties,
                avroDeserializer,
                dataProcessingService,
                i + 1
            );
            consumers.add(consumer);
            consumerExecutor.submit(consumer::start);
        }
    }
}
```

3.2 NetflowConsumer - Individual Consumer Implementation

File: `src/main/java/com/nimbo/ingester/consumer/NetflowConsumer.java`

Each consumer instance handles a subset of Kafka partitions as assigned by the consumer group. The consumer implements:

Core Components:

Component	Purpose
KafkaConsumer	Polls records from assigned partitions
Writer Thread Pool	Parallel writers for HDFS (configurable threads)
Rebalance Listener	Flushes buffered records before partition revocation

Component	Purpose
Graceful Shutdown	Flushes remaining records on container termination

Consumer Loop Flow:

```
1. Subscribe to topic with rebalance listener
2. Poll records from Kafka
3. Submit records to writer thread pool
4. Wait for all writers to complete
5. Flush batch to HDFS
6. Commit offsets only after successful flush
7. Repeat
```

Consumer Configuration:

```yaml
ingester:
  kafka:
    consumer-instances: 3          # Number of consumer threads
    writer-threads-per-instance: 4 # Writer threads per consumer
    writer-queue-capacity: 10000   # Queue size for writer tasks
    enable-auto-commit: false      # Manual commit for zero data loss
    max-poll-records: 500          # Records per poll
    poll-timeout-ms: 1000          # Poll timeout
```

3.3 KafkaProperties - Externalized Configuration

File: `src/main/java/com/nimbo/ingester/config/KafkaProperties.java`

All Kafka and consumer configuration is externalized using Spring Boot's `@ConfigurationProperties` with validation.

```java
@ConfigurationProperties(prefix = "ingester.kafka")
public class KafkaProperties {
    @NotBlank
    private String bootstrapServers;
    @NotBlank
    private String topic;
    @NotBlank
    private String consumerGroup;
    private boolean enableAutoCommit = false;
    @Min(1)
    private int consumerInstances = 1;
    @Min(1)
    private int writerThreadsPerInstance = 2;
```

```
    // ... additional configs
}
```

---

4. Module 2: Avro Deserialization

4.1 AvroDeserializer

File: `src/main/java/com/nimbo/ingester/deserializer/AvroDeserializer.java`

The `AvroDeserializer` converts binary Avro data from Kafka into `GenericRecord` objects.

Implementation Details:

- Uses Avro's `GenericDatumReader` for schema-based deserialization
- Thread-safe (stateless) design
- Schema is loaded once at startup from `NetFlowV2Record.avsc`
- Handles deserialization failures with custom `DeserializationException`

```
public GenericRecord deserialize(byte[] data) {
    BinaryDecoder decoder = decoderFactory.binaryDecoder(data, null);
    return reader.read(null, decoder);
}
```

4.2 Avro Schema

File: `src/main/resources/avro/NetFlowV2Record.avsc`

The Avro schema defines the Netflow record structure with 43 fields including:

- Network layer fields: `IPV4_SRC_ADDR`, `IPV4_DST_ADDR`, `PROTOCOL`
- Transport layer fields: `L4_SRC_PORT`, `L4_DST_PORT`
- Traffic metrics: `IN_BYTES`, `OUT_BYTES`, `IN_PKTS`, `OUT_PKTS`
- Flow metadata: `FLOW_DURATION_MILLISECONDS`, `TCP_FLAGS`
- Application layer: `DNS_QUERY_ID`, `DNS_QUERY_TYPE`, `FTP_COMMAND_RET_CODE`

---

5. Module 3: Data Processing Service

5.1 DataProcessingService - Orchestration Layer

File: `src/main/java/com/nimbo/ingester/service/DataProcessingService.java`

The `DataProcessingService` acts as the bridge between the consumer and the storage layer, providing a clean separation of concerns.

Responsibilities:

- Receives `IngestedRecord` from consumer
- Delegates to `NetFlowStorage.accept()`
- Returns offsets to commit after successful persistence
- Handles flush operations for graceful shutdown and rebalancing

```
@Service
public class DataProcessingService {
    public Optional<Map<TopicPartitionKey, Long>> processRecord(IngestedRecord record) {
        return netFlowStorage.accept(record);
    }

    public Map<TopicPartitionKey, Long> flush() {
        return netFlowStorage.flush();
    }
}
```

5.2 IngestedRecord - Data Model

File: `src/main/java/com/nimbo/ingester/model/IngestedRecord.java`

The `IngestedRecord` is a Java Record that wraps an Avro `GenericRecord` with Kafka metadata.

```
public record IngestedRecord(
    GenericRecord record,
    String topic,
    int partition,
    long offset,
    int serializedSizeBytes
) {
    public IngestedRecord {
        Objects.requireNonNull(record, "record must not be null");
        Objects.requireNonNull(topic, "topic must not be null");
        if (serializedSizeBytes < 0) {
            throw new IllegalArgumentException("serializedSizeBytes must not be negative");
        }
    }
}
```

Purpose:

- Holds the actual Avro data (`GenericRecord`)
- Maintains Kafka metadata for offset management

- Tracks serialized size for batch accumulation decisions

---

6. Module 4: Batch Accumulation

6.1 BatchAccumulator

File: `src/main/java/com/nimbo/storage/service/batching/BatchAccumulator.java`

The `BatchAccumulator` buffers incoming records until a flush condition is met, enabling efficient batch writes to HDFS.

Key Features:

Feature	Implementation
Buffer	`List<IngestedRecord>` for accumulated records
Size Tracking	Accumulates serialized size in bytes
Time Tracking	Tracks age of first record in batch
Offset Tracking	Maintains pending offsets per partition
Thread Safety	Not thread-safe (single consumer per instance)

State Management:

```
private final List<IngestedRecord> buffer = new ArrayList<>();
private Instant firstRecordAt;
private long accumulatedSizeBytes;
private final Map<TopicPartitionKey, Long> pendingOffsets = new HashMap<>();
```

6.2 BatchPolicy - Flush Triggers

File: `src/main/java/com/nimbo/storage/service/batching/BatchPolicy.java`

The `BatchPolicy` determines when a batch should be flushed based on three configurable thresholds:

Threshold	Default	Description
Max Record Count	50,000	Flush when batch reaches this many records

Threshold	Default	Description
Max Size Bytes	64 MB	Flush when batch reaches this size
Max Age	2 minutes	Flush when oldest record exceeds this age

```
public boolean shouldFlush(int recordCount, long totalSizeBytes,
                           Instant firstRecordAt, Instant now) {
    if (recordCount >= maxRecordCount) return true;
    if (totalSizeBytes >= maxSizeBytes) return true;
    return Duration.between(firstRecordAt, now).compareTo(maxBatchAge) >= 0;
}
```

6.3 RecordBatch - Immutable Batch Representation

File: `src/main/java/com/nimbo/storage/service/batching/RecordBatch.java`

The `RecordBatch` is an immutable snapshot of the accumulator's state at flush time.

```
public final class RecordBatch {
    private final List<IngestedRecord> records;
    private final Instant createdAt;
    private final long totalSizeBytes;

    public Map<TopicPartitionKey, Long> getOffsetsToCommit() {
        Map<TopicPartitionKey, Long> commitOffsets = new HashMap<>();
        for (IngestedRecord rec : records) {
            TopicPartitionKey key = new TopicPartitionKey(rec.topic(), rec.partition());
            commitOffsets.merge(key, rec.offset() + 1, Math::max);
        }
        return Map.copyOf(commitOffsets);
    }
}
```

---

7. Module 5: Storage Layer (HDFS & Parquet)

7.1 NetFlowStorage - Storage Orchestration

File: `src/main/java/com/nimbo/storage/service/NetFlowStorage.java`

The `NetFlowStorage` service orchestrates the complete storage flow from batch accumulation to HDFS persistence.

Flow:

```
1. accept(record) → Add to BatchAccumulator
2. if shouldFlush() → flushInternal()
3. Create temp file (.tmp)
4. Write Parquet via ParquetFileWriter
5. Atomic rename to final file
6. Return offsets to commit
7. Clear accumulator
```

7.2 FileNameGenerator - File Naming Convention

File: `src/main/java/com/nimbo/storage/service/naming/FileNameGenerator.java`

Generates unique Parquet filenames with partition, timestamp, and UUID to prevent collisions.

```
public String generateFileName(int partition) {
    String timestamp = DateTimeFormatter.ISO_INSTANT
        .format(Instant.now())
        .replace(":", "-");
    return String.format("netflow-p%d-%s-%s.parquet",
        partition, timestamp, UUID.randomUUID());
}

public String generateTempFileName(String finalFileName) {
    return "." + finalFileName + ".tmp";
}
```

Example:

- Final: `netflow-p0-2026-08-13T08-30-15Z-abc123.parquet`
- Temp: `.netflow-p0-2026-08-13T08-30-15Z-abc123.parquet.tmp`

7.3 ParquetFileWriter - Parquet Serialization

File: `src/main/java/com/nimbo/storage/service/parquet/ParquetFileWriter.java`

The `ParquetFileWriter` handles the actual serialization of Avro records to Parquet format.

Features:

- Uses AvroParquetWriter for schema-aware writing
- Configurable compression (SNAPPY default)
- Configurable row group and page sizes
- Atomic file creation with temporary file first

```
public void writeToPath(RecordBatch batch, Path targetPath) {
    Schema schema = extractSchema(batch);
    try (ParquetWriter<GenericRecord> writer = AvroParquetWriter
            .<GenericRecord>builder(targetPath)
            .withSchema(schema)
            .withCompressionCodec(codec)
            .withRowGroupSize(blockSizeBytes)
            .withPageSize(pageSizeBytes)
            .build()) {
        for (IngestedRecord ingestedRecord : batch.getRecords()) {
            writer.write(ingestedRecord.record());
        }
    }
}
```

7.4 HdfsFileStorage - HDFS File Operations

File: `src/main/java/com/nimbo/storage/service/hdfs/HdfsFileStorage.java`

The `HdfsFileStorage` implementation handles low-level HDFS file operations.

Operations:

Operation	Purpose
`ensureDirectoryExists()`	Creates HDFS directories if they don't exist
`commitAtomic()`	Atomic rename from temp to final file
`cleanupTempFile()`	Deletes temporary files on failure

```
public void commitAtomic(Path tempPath, Path finalPath) {
    if (fileSystem.exists(finalPath)) {
        throw new HdfsStorageException("Target file already exists: " + finalPath);
    }
    boolean renamed = fileSystem.rename(tempPath, finalPath);
    if (!renamed) {
        throw new HdfsStorageException("Failed to rename " + tempPath + " -> " + finalPath);
    }
}
```

7.5 StorageConfiguration - HDFS Client Setup

File: `src/main/java/com/nimbo/storage/config/StorageConfiguration.java`

Configures the HDFS `FileSystem` client with support for HA Namenodes.

Features:

- High Availability Namenode configuration
- Dynamic node resolution via `nameservice`
- Failover proxy provider for automatic failover
- Configurable HDFS user

```
@Bean
public FileSystem fileSystem(HdfsProperties hdfsProperties) {
    Configuration conf = new Configuration();
    String nameservice = hdfsProperties.getNameservice();
    if (nameservice != null && !nameservice.isBlank()) {
        conf.set("dfs.nameservices", nameservice);
        Map<String, String> nodes = hdfsProperties.getNodes();
        if (nodes != null && !nodes.isEmpty()) {
            String nodeAliases = String.join(",", nodes.keySet());
            conf.set("dfs.ha.namenodes." + nameservice, nodeAliases);
            for (Map.Entry<String, String> entry : nodes.entrySet()) {
                conf.set("dfs.namenode.rpc-address." + nameservice + "." + entry.getKey(),
                    entry.getValue());
            }
        }
        conf.set("dfs.client.failover.proxy.provider." + nameservice,
            "org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider");
    }
    return FileSystem.get(URI.create(hdfsProperties.getUri()), conf, user);
}
```

---

8. Configuration Management

8.1 Configuration Hierarchy

```
application.yml (Base configuration)
    ↓
application-dev.yml (Dev-specific overrides)
    ↓
application-prod.yml (Production-specific overrides)
    ↓
Environment Variables (Runtime overrides)
```

8.2 Configuration Files

application.yml - Base Configuration:

```
ingester:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:10.10.61.51:9092,10.10.61.53:9092,10.10.61.54:90
    topic: ${KAFKA_TOPIC:netflow}
    consumer-group: ${KAFKA_CONSUMER_GROUP:netflow-java-consumer8}
    enable-auto-commit: false
    poll-timeout-ms: 1000
    max-poll-records: 500
    session-timeout-ms: 30000
    heartbeat-interval-ms: 3000
    max-poll-interval-ms: 300000
    consumer-instances: ${CONSUMER_INSTANCES:1}
    writer-threads-per-instance: ${WRITER_THREADS:2}
    writer-queue-capacity: ${WRITER_QUEUE_CAPACITY:1000}
    batch-max-records: ${BATCH_MAX_RECORDS:50000}
    batch-max-size-bytes: ${BATCH_MAX_SIZE_BYTES:67108864}
    batch-max-age-minutes: ${BATCH_MAX_AGE_MINUTES:2}
```

application-dev.yml - Development:

```
ingester:
  kafka:
    consumer-instances: 1
    writer-threads-per-instance: 2
    writer-queue-capacity: 100
    batch-max-records: 1000
    batch-max-size-bytes: 1048576
    batch-max-age-minutes: 1
```

application-prod.yml - Production:

```
ingester:
  kafka:
    consumer-instances: 3
    writer-threads-per-instance: 4
    writer-queue-capacity: 10000
    batch-max-records: 100000
    batch-max-size-bytes: 134217728
    batch-max-age-minutes: 5
```

8.3 Environment Variables

Variable	Purpose	Default
`KAFKA_BOOTSTRAP_SERVERS`	Kafka broker addresses	10.10.61.51:9092,...

Variable	Purpose	Default
`KAFKA_TOPIC`	Source topic name	netflow
`KAFKA_CONSUMER_GROUP`	Consumer group ID	netflow-java-consumer8
`HDFS_URI`	HDFS Namenode URI	hdfs://team1
`HDFS_USER`	HDFS user	team1
`HDFS_BASE_PATH`	HDFS base path	/netflow/parquet
`CONSUMER_INSTANCES`	Number of consumer instances	1
`WRITER_THREADS`	Writer threads per consumer	2

---

9. Exception Handling

9.1 Exception Hierarchy

```
RuntimeException
    ├── KafkaConsumerException
    ├── DeserializationException
    └── StorageException
        ├── HdfsStorageException
        └── ParquetWriteException
```

9.2 Error Handling Strategy

Scenario	Handling
Deserialization Failure	Log error, skip record, continue
HDFS Write Failure	Log error, keep buffer for retry
Kafka Connection Failure	Retry with exponential backoff
Consumer Rebalance	Flush and commit before revocation

Scenario	Handling
Shutdown	Flush remaining buffer before exit

---

10. Zero Data Loss Guarantee

10.1 The Problem

Without manual offset management:

```
1. Read record (offset=42)
2. auto-commit commits offset=42
3. 💥 HDFS write fails
4. offset=42 is committed but data is NOT in HDFS
5. ❌ DATA LOSS!
```

10.2 The Solution

With manual offset commit:

```
1. Read record (offset=42)
2. Store in batch accumulator
3. HDFS write succeeds → returns offset=42
4. commitSync() commits ONLY offset=42
5. ✅ Data is in HDFS AND offset is committed
6. 💥 Crash → no data loss (uncommitted offsets are re-consumed)
7. ✅ Zero data loss guarantee!
```

10.3 Implementation

```
// Process record and get offsets to commit
for (ConsumerRecord<byte[], byte[]> record : records) {
    Optional<Map<TopicPartitionKey, Long>> result = processRecord(record);
    result.ifPresent(offsets -> offsetsToCommit.putAll(offsets));
}

// Only commit offsets that were successfully persisted to HDFS
if (!offsetsToCommit.isEmpty()) {
    kafkaConsumer.commitSync(commitMap);
```

```
    log.debug("Committed offsets: {}", offsetsToCommit);
}
```

10.4 Flush on Graceful Shutdown

```
@PreDestroy
public void shutdown() {
    Map<TopicPartitionKey, Long> flushedOffsets = dataProcessingService.flush();
    if (!flushedOffsets.isEmpty()) {
        commitMap.putAll(flushedOffsets);
        kafkaConsumer.commitSync(commitMap);
    }
}
```

---

11. Scalability Architecture

11.1 Two-Dimensional Scaling

Dimension 1: Horizontal Scaling (More Consumers)

Consumers	Partitions	Throughput
1	3	Baseline
3	9	3x baseline
6	18	6x baseline

Dimension 2: Vertical Scaling (More Writer Threads)

Consumers	Writers per Consumer	Total Writers
1	2	2
3	2	6
3	4	12

11.2 Scaling Formula

```
Total Parallel Processing = CONSUMER_INSTANCES × WRITER_THREADS_PER_INSTANCE

Example: 3 consumers × 4 writers = 12 parallel threads
- 3 consumers pulling from Kafka
- 12 writers writing to HDFS in parallel
- Up to 12x faster than single consumer + single writer
```

---

12. Infrastructure Configuration

12.1 Server Discovery

Server	Service	Port	Role
10.10.61.51	Kafka Broker	9092	Partition 1 Leader
10.10.61.52	HDFS Namenode	9000, 9870	Partition 2 Leader + Namenode
10.10.61.53	Kafka Broker + Zookeeper	9092, 2181	Replica + Zookeeper
10.10.61.54	Kafka Broker	9092	Partition 0 Leader

12.2 Kafka Topic Configuration

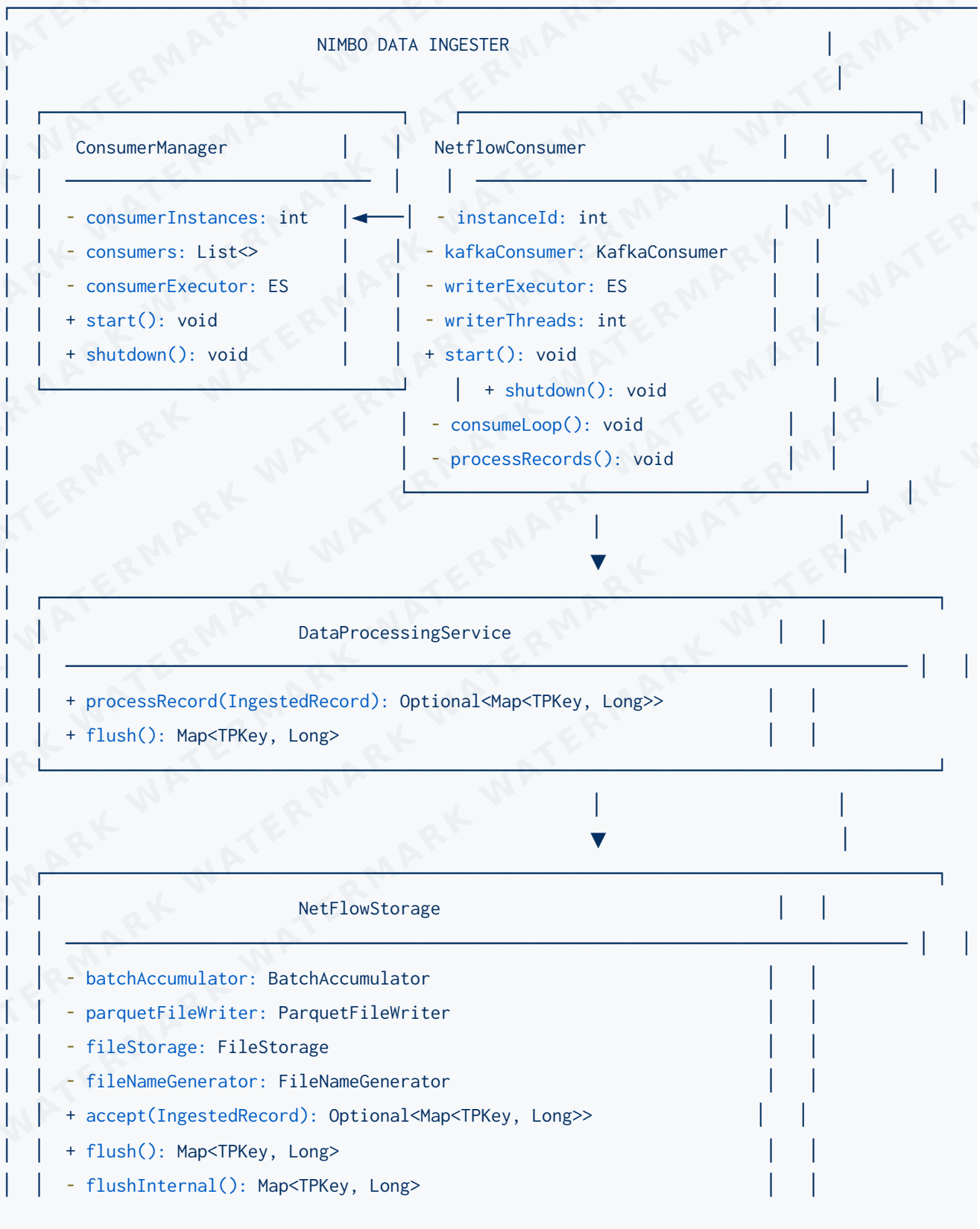
Setting	Value
Topic	netflow
Partitions	3
Replication Factor	2
Retention	30 days
Consumer Group	netflow-java-consumer8

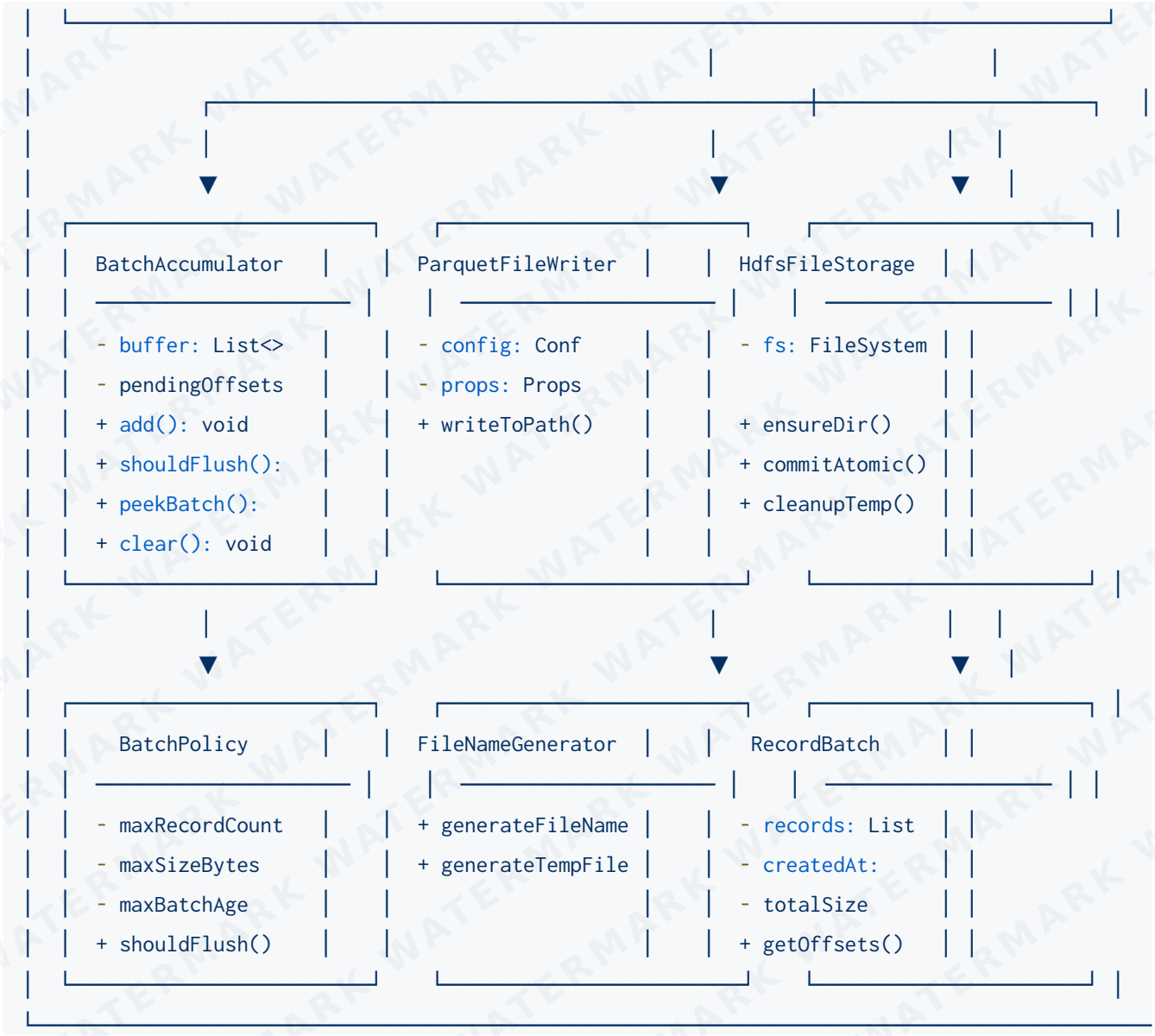
12.3 HDFS HA Configuration

```
hdfs://team1 (Nameservice)
├── team1-node1 (10.10.61.51:8020)  ← HA Namenode 1
```

```
├── team1-node2 (10.10.61.52:8020)  ← HA Namenode 2
└── team1-node3 (10.10.61.53:8020)  ← HA Namenode 3
```

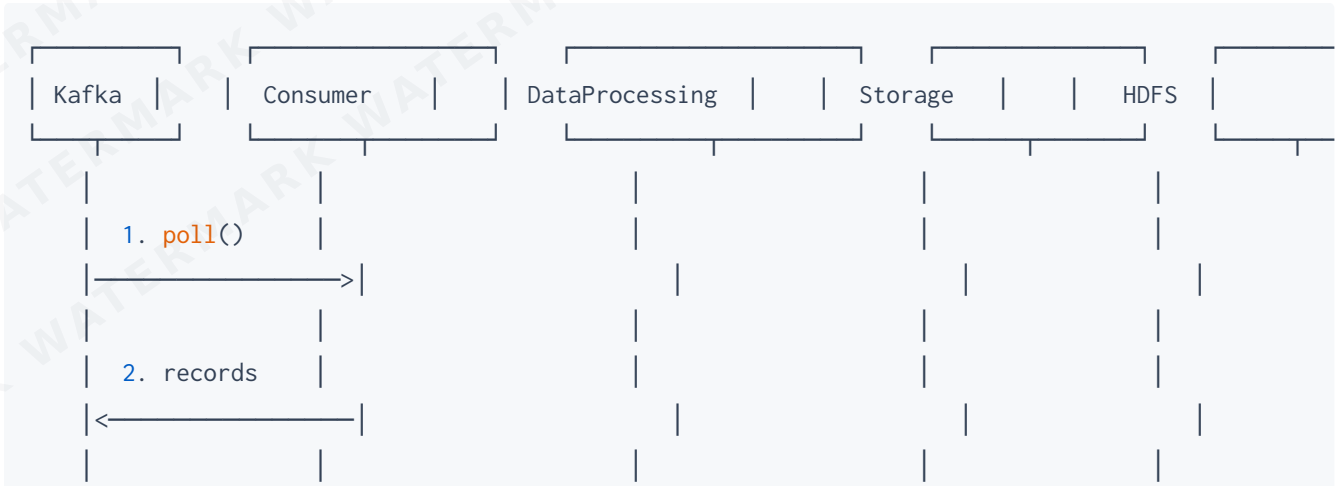
13. Class Diagram

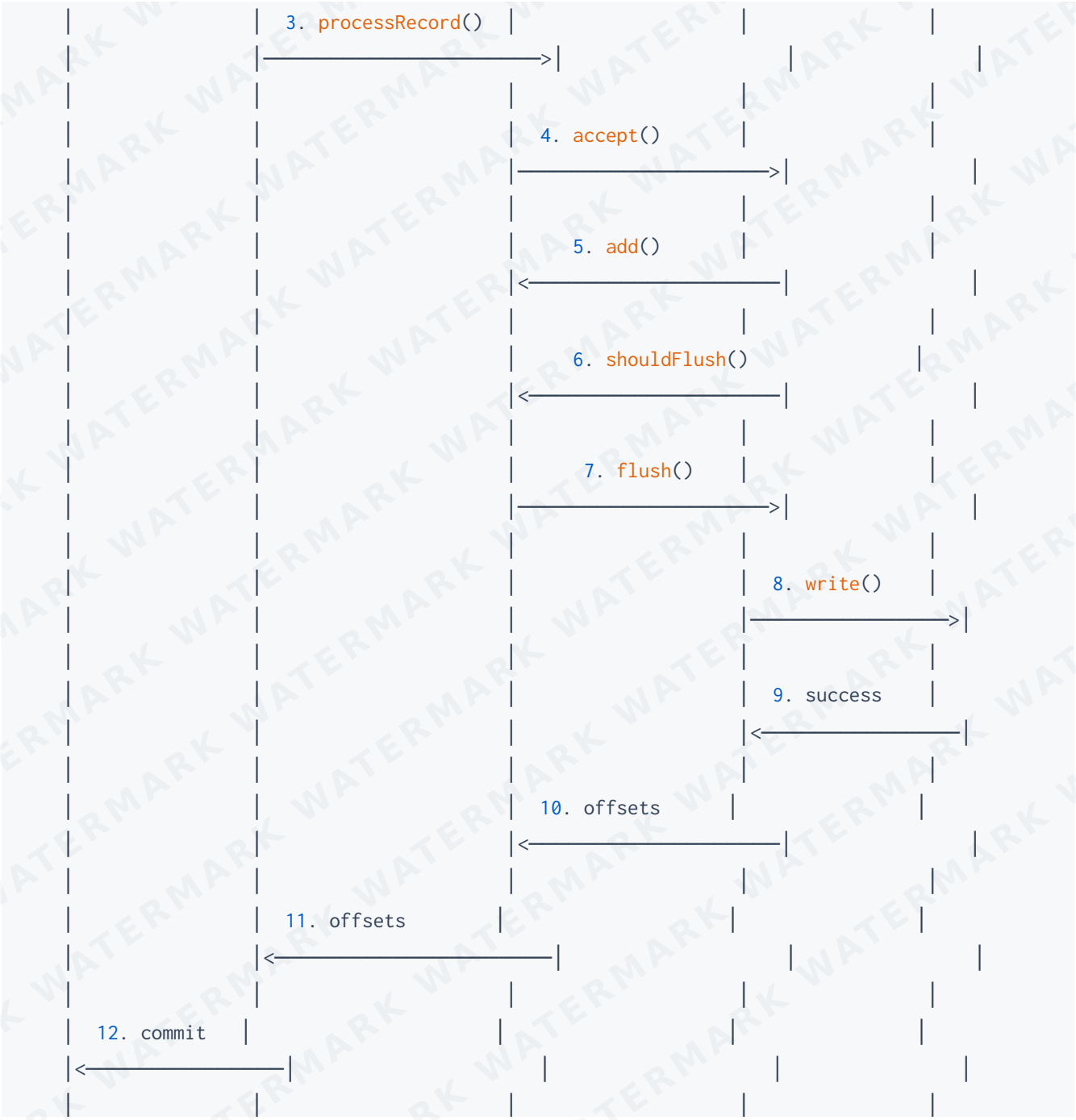




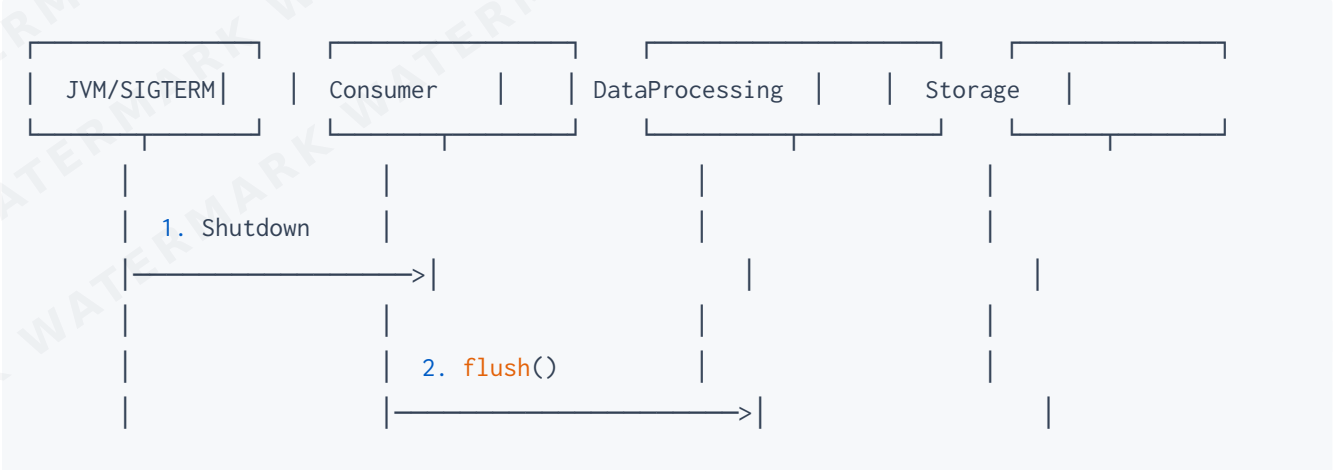
14. Sequence Diagrams

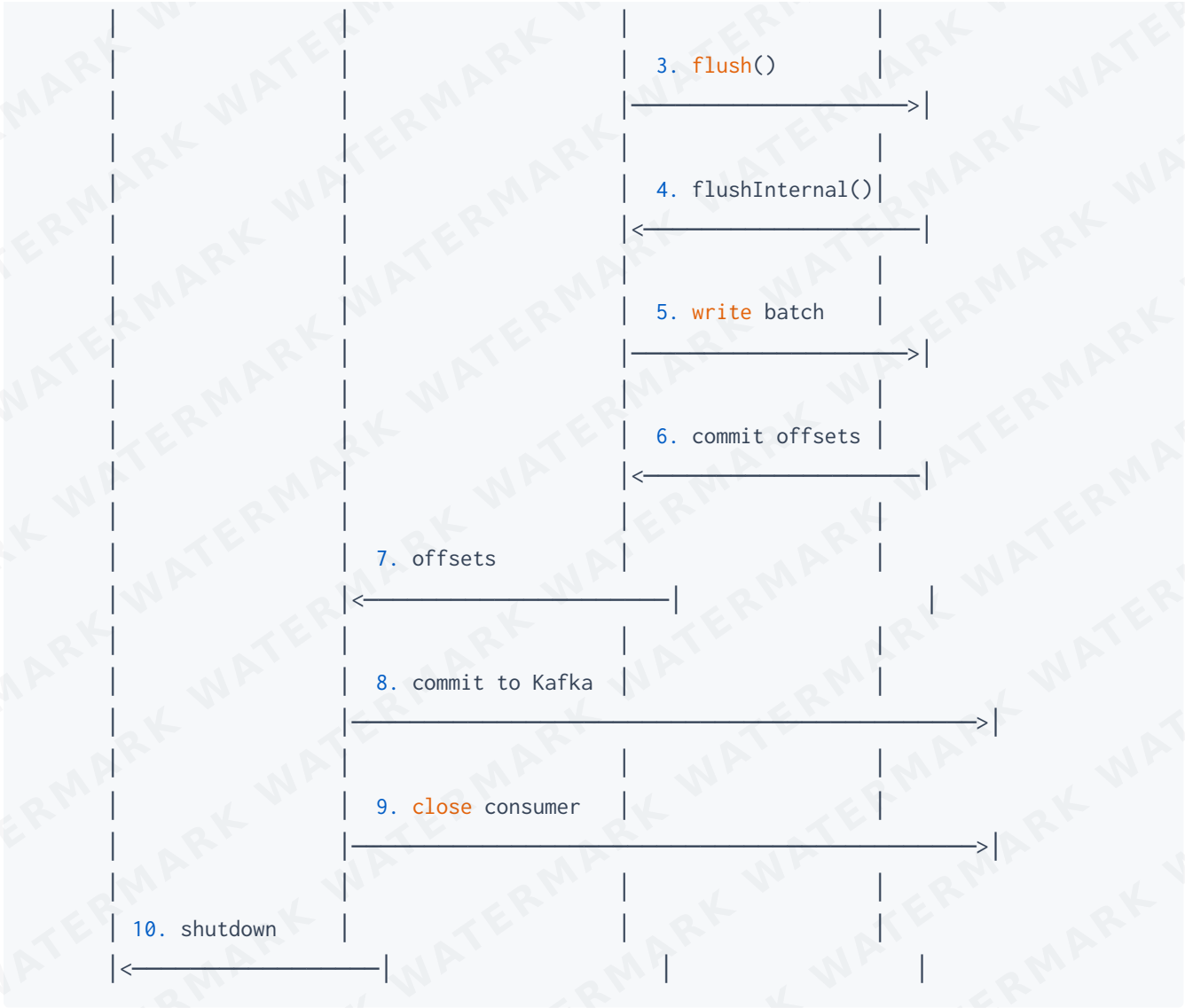
14.1 Message Processing Flow



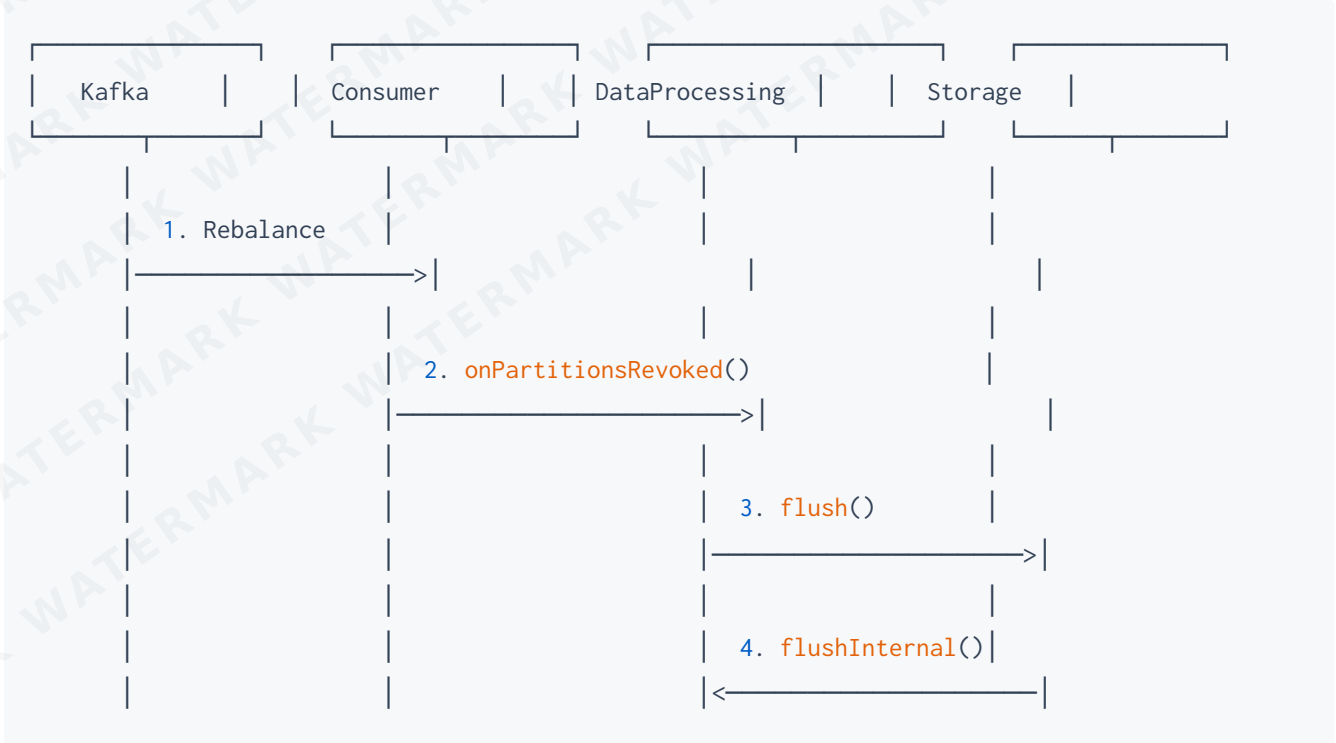


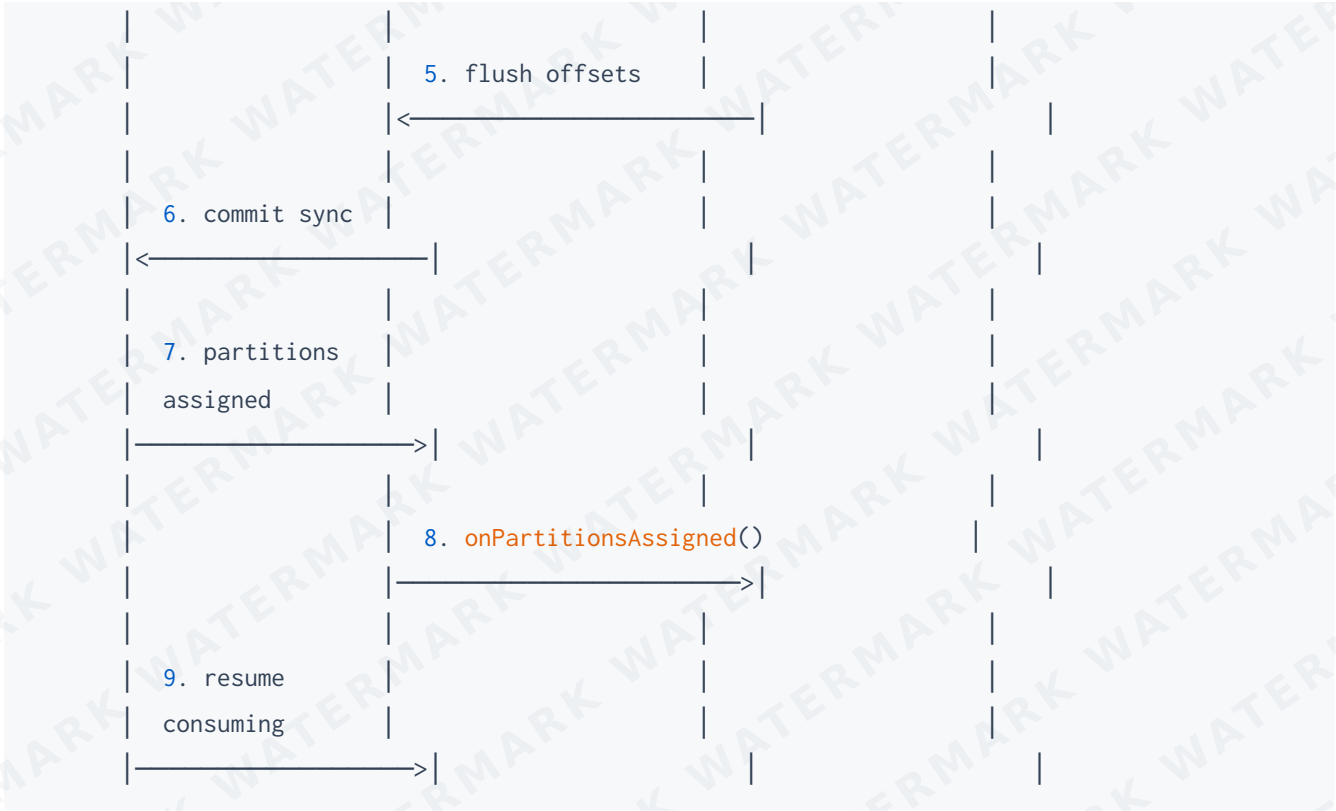
14.2 Graceful Shutdown Flow





14.3 Consumer Rebalance Flow





15. Testing Strategy

15.1 Unit Tests

Test Class	Coverage
`KafkaPropertiesTest`	Configuration validation, default values
`AvroDeserializerTest`	Avro deserialization logic
`DataProcessingServiceTest`	Service orchestration with mocks
`NetflowConsumerTest`	Consumer lifecycle and shutdown
`ConsumerManagerTest`	Multi-instance management
`BatchPolicyTest`	Batch threshold calculations
`BatchAccumulatorTest`	Batch accumulation logic
`RecordBatchTest`	Immutable batch behavior

15.2 Integration Tests

Test Class	Technology	Purpose
`StorageIT`	Testcontainers	End-to-end storage flow
`SmokeTest`	Testcontainers	Full pipeline validation

15.3 Testing Tools

Tool	Purpose
JUnit 5	Test framework
Mockito	Mocking for unit tests
AssertJ	Fluent assertions
Awaitility	Asynchronous test waiting
Testcontainers	Integration test containers

---

16. Performance Considerations

16.1 Optimization Decisions

Decision	Rationale
Batch Accumulation	Reduces HDFS write operations (50000 records/batch vs 1 record)
Parallel Writers	Multiple threads write simultaneously to HDFS
Compression	SNAPPY compression reduces storage and network I/O
Atomic File Operations	Prevents partial file writes on failure
Manual Offset Commits	Eliminates duplicate message processing

16.2 Resource Configuration

Resource	Recommended
Heap Size	512MB - 1GB per consumer instance
Writer Threads	2-4 per consumer instance
Batch Size	50,000 - 100,000 records
Batch Age	2-5 minutes

---

17. Error Handling & Monitoring

17.1 Logging Levels

Level	Component	When Used
INFO	Application startup, consumer lifecycle	Major events
DEBUG	Record processing, offset commits	Development troubleshooting
WARN	Kafka connectivity issues, HDFS retries	Non-critical issues
ERROR	Deserialization failures, write failures	Critical failures

17.2 Health Checks

Spring Boot Actuator endpoints exposed:

Endpoint	Purpose
`/actuator/health`	Application health status
`/actuator/info`	Application information
`/actuator/metrics`	Performance metrics

17.3 Key Metrics

Metric	Description
Consumer Lag	Difference between log end offset and consumer offset
Batch Size	Number of records per batch
Batch Flush Frequency	How often batches are flushed
HDFS Write Latency	Time to write a batch to HDFS
Parquet File Size	Size of generated Parquet files

---

18. Security Considerations

Concern	Implementation
Secrets Management	No secrets in code, use environment variables
HDFS Authentication	Uses `HDFS_USER` configuration
Kafka Security	PLAINTEXT for lab environment (configurable)
Non-Root User	Container runs as `appuser`, not root
Input Validation	Avro schema validation on deserialization

---

19. Known Limitations & Future Improvements

19.1 Current Limitations

Limitation	Impact
Single JVM	All consumers in one JVM (can be scaled by running multiple containers)
No Circuit Breaker	Failures cascade without resilience patterns

Limitation	Impact
No Backpressure	No throttling mechanism for high data volume
Static Configuration	Some changes require restart

19.2 Future Improvements

Improvement	Priority	Description
Circuit Breakers	Medium	Add resilience patterns for HDFS failures
Dynamic Scaling	Medium	Auto-scaling based on consumer lag
Metrics Pipeline	Medium	Prometheus + Grafana integration
Kubernetes Operator	Low	Native K8s scaling and management
Avro Schema Evolution	Low	Support for schema registry and evolution

---

20. Technical Decisions Summary

Decision	Rationale
Manual Offset Commits	Zero data loss guarantee
Batch Accumulation	Reduce HDFS write operations
Multi-Consumer Architecture	Horizontal scalability
Multi-Threaded Writers	Vertical scalability
Atomic File Operations	Prevent partial writes
Spring Boot	Production-ready framework with configuration management

Decision	Rationale
Avro Serialization	Schema evolution and compact binary format
Parquet Storage	Columnar format for analytics queries
HDFS HA	No single point of failure
Externalized Configuration	Environment-specific configs without code changes

---

This report documents the complete implementation of the Nimbo Data Ingester as of Week 1 of the project. The architecture provides a solid foundation for the subsequent phases of the project, with built-in scalability, reliability, and zero data loss guarantees.

WATERMARK